B — Skill libraries and program synthesis (the Skill tier)

0. Role in the system

This pack informs the **Skill tier**: skills stored as verified, parameterized, tested code with declared inputs/outputs (contracts); retrieval by embedding similarity over natural-language descriptions; composition of skills by a small planner (and by code calling code); library growth, deduplication and retirement; and the symbolic ancestor of the idea, Soar's chunking (a solved subgoal is compiled into a reusable production). After reading this pack the implementing agents must be able to build: (1) the **admission loop** that turns a verified worker/senior trajectory into a skill record (Voyager + LATM: propose, execute, test, describe, embed, store); (2) the **retriever** (embed task context + environment feedback, top-k by cosine similarity, inject into the prompt); (3) the **router/dispatcher** that decides "existing skill vs. request a new one" (LATM); (4) the **consolidation pass** run at idle time that refactors the library under a compression (MDL) objective and retires drifting skills (DreamCoder abstraction sleep + SoK lifecycle); (5) the **evaluation harness** that proves skills help (SkillsBench-style no-skill / curated / self-generated comparison with deterministic verifiers). The action space for executing skills is executable code (CodeAct), not JSON tool calls.

1. Papers

1.1 Voyager: An Open-Ended Embodied Agent with Large Language Models (Wang et al., 2023) — arXiv:2305.16291

Problem and claim. Lifelong, open-ended learning in Minecraft without human intervention. Claim: an LLM-driven agent with (i) an automatic curriculum that maximizes exploration, (ii) an ever-growing skill library of executable code, and (iii) an iterative prompting mechanism fed by environment feedback, execution errors and self-verification can keep acquiring compositional skills, with GPT-4 queried as a black box (no fine-tuning). Published in TMLR 2024.

Headline results (MineDojo, vs. ReAct, Reflexion, AutoGPT; Section 3.3). - Unique items discovered: **63 within 160 prompting iterations**, **3.3x** more than baselines. - Map traversal: **2.3x** longer distances. - Tech tree (Table 1; successes out of 3, mean prompting iterations +- spread):

Method	Wooden	Stone	Iron	Diamond
ReAct	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
Reflexion	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT	92 +- 72 (3/3)	94 +- 72 (3/3)	135 +- 103 (3/3)	N/A (0/3)
Voyager w/o skill library	7 +- 2 (3/3)	9 +- 4 (3/3)	29 +- 11 (3/3)	N/A (0/3)
Voyager	6 +- 2 (3/3)	11 +- 2 (3/3)	21 +- 7 (3/3)	102 (1/3)

Speedups stated: wooden **15.3x**, stone **8.5x**, iron **6.4x**; only Voyager unlocks diamond. - Zero-shot generalization to a new world, cleared inventory, 50-iteration cap (Table 2): Voyager **19 +- 3 (3/3)** diamond pickaxe, **18 +- 7 (3/3)** golden sword, **21 +- 5 (3/3)** lava bucket, **18 +- 2 (3/3)** compass. Voyager w/o skill library: 36 (2/3), 30 +- 9 (3/3), 27 +- 9 (3/3), 26 +- 3 (3/3). AutoGPT w/ Voyager's skill library: 39 (1/3), 30 (1/3), N/A (0/3), 30 (2/3). ReAct/Reflexion/AutoGPT alone: 0/3 everywhere.

Method — equations. None (the paper is procedural; no objective is written).

Method — algorithm.

Appendix A.1 pseudocode (transcribed in structure; the paper's exact variable names are paraphrased where the fetch did not quote them):

```
voyager(env, curriculum_agent, action_agent, critic_agent, skill_manager):
  loop forever:
    exploration_progress = curriculum_agent.get_exploration_progress(
        curriculum_agent.completed_tasks, curriculum_agent.failed_tasks)
    task = curriculum_agent.propose_next_task(agent_state, exploration_progress)
    code = None; env_feedback = None; execution_errors = None; critique = None; success = False
    for i in 1..4:                       # "try at most 4 rounds before moving on to the next task"
        skills = skill_manager.retrieve_skills(task, env_feedback)
        code = action_agent.generate_code(task, code, env_feedback, execution_errors,
                                          critique, skills)
        agent_state, env_feedback, execution_errors = env.step(code)
        success, critique = critic_agent.check_task_success(task, agent_state)
        if success: break
    if success:
        skill_manager.add_skill(code)
        curriculum_agent.add_completed_task(task)
    else:
        curriculum_agent.add_failed_task(task)
```

1. **Automatic curriculum (Section 2.1, A.3).** Prompt components: (a) directives: “discover as many diverse things as possible” and the next task must not be too hard; (b) agent state: inventory, equipment, nearby blocks, other recently seen blocks, nearby entities, chests, biome, time, health bar, hunger bar, position; (c) previously completed and failed tasks; (d) additional context; (e) chain-of-thought instruction. GPT-4 returns the next task; temperature 0.1 (all other calls temperature 0). *Additional context* (A.3.2): GPT-3.5 self-asks **at least 5 and at most 10** questions, each paired with a “concept”; the concept retrieves the most relevant wiki document (optional); GPT-3.5 answers each question (“Answer: Unknown” if it does not know). Example pair: “How to make a wooden pickaxe?” / concept “wooden pickaxe”. *Warm-up schedule* (Table A.1, keyed to **number of completed tasks**): core inventory (log, planks, stick, crafting table, furnace, dirt, coal, pickaxe, sword, axe), equipment, nearby blocks, position: after 0; nearby entities: 5; full inventory: 7; other recently seen blocks, biome: 10; health bar, hunger bar, time, additional context: 15.
2. **Skill library (Section 2.2, A.4).** A skill is an `async function name(bot) { ... }` in Mineflayer JavaScript. On success, GPT-3.5 writes a description of the program (Prompt 5: do not mention the function name, do not mention `bot.chat` or helper functions, describe only the main function, at most 6 sentences, single line). Key = embedding of the description (text-embedding-ada-002); value = the program; stored in a vector DB. Retrieval: GPT-3.5 drafts a general suggestion for solving the new task; that suggestion plus environment feedback is the query; **top-5** skills are retrieved and placed in the code-generation prompt directly after the Mineflayer API list. New skills call older ones by name (e.g. `craftWoodenPlanks(bot)` calls `mineWoodLog(bot)` and `craftItem(bot, plankName, 1)`).
3. **Iterative prompting (Section 2.3, A.4.1).** Code-generation prompt components (12): guidelines; authors’ control primitives `exploreUntil, mineBlock, craftItem, placeItem, smeltItem, killMob, getItemFromChest, depositItemIntoChest`; Mineflayer primitives (`bot.pathfinder.goto, bot.equip, bot.fish`, ...); retrieved skills; code from last round; environment feedback (chat log); execution errors; critique from self-verification; agent state; the task; task context (GPT-3.5 suggestions); chain-of-thought. Required response format: **Explain** (why the last code failed) / **Plan** (numbered steps) / **Code** (helper functions then one main async function). Guidelines (A.4.2): async function taking only `bot`; reuse helper programs; be generic and reusable, check required items first; do not reuse functions from “code from the last round” (not saved); define variables inside functions; call `bot.chat` for progress; use `exploreUntil` with a random direction; `maxDistance` 32 for `findBlocks`/`findBlock`; no infinite loops or recursion; no `bot.on`/`bot.once`; meaningful function name. Feedback: environment feedback via `bot.chat()` inside primitives (e.g. “cannot craft iron chestplate, need 7 more iron ingots”); execution errors from the interpreter; self-verification by a separate GPT-4 critic given agent state + task, returning success and, on failure, a critique. Stuck after **4 rounds** -> curriculum proposes another task.
4. **Code (github.com/MineDojo/Voyager, `voyager/agents/skill.py`, `voyager/voyager.py`, `voyager/prompts/critic.txt`).** `SkillManager(model_name="gpt-3.5-turbo", temperature=0, retrieval_top_k=5, ckpt_dir="ckpt")`; Chroma collection `skill_vectordb` with `OpenAIEmbeddings()`; `add_new_skill(info)` with `info = {task, program_name, program_code}` generates the description, handles name collisions by deleting the old vector entry and saving the file as `{name}V{i}` (i from 2), calls `vectordb.add_texts(texts=[description], ids=[program_name], metadatas=[{"name": program_name}])`, writes `skill/code/{name}.js`, `skill/description/{name}.txt`, `skill/skills.json`; `retrieve_skills(query)`: `k = min(count, retrieval_top_k)`, `similarity_search_with_score(query, k)`, returns the code strings. In `Voyager.step()` the query is `context + "\n\n" + action_agent.summarize_chatlog(events)`;

`action_agent_task_max_retries=4`; `max_iterations=160`; `openai_api_request_timeout=240`. Critic output is JSON `{"reasoning": str, "success": bool, "critique": str}` with seven worked examples.

Hyperparameters and setup. gpt-4-0314 for code generation, curriculum and critic; gpt-3.5-turbo-0301 for QA, skill descriptions and retrieval suggestions; text-embedding-ada-002; temperature 0 except curriculum 0.1; 3 trials per experiment; 160 iterations (tech tree), 50 (generalization); MineDojo + Mineflayer; system and user prompts concatenated into a single prompt (A.2). “Prompting iteration” is not defined explicitly (not in fetched text — verify in paper Section 3.1).

Ablations that matter (Section 3.4, Figure 9). Random curriculum: **-93%** discovered items; manual curriculum: worse than automatic. No skill library: plateau in later stages (Table 1: iron 29 +- 11 vs 21 +- 7). No self-verification: **-73%** items (most important feedback). GPT-4 vs GPT-3.5 for code generation: **5.7x** more unique items. Each of environment feedback / execution errors / self-verification is needed (Figure 9 right).

Code / data / license. https://github.com/MineDojo/Voyager (MIT); project page https://voyager.minedojo.org; Python >= 3.9, Node >= 16.13, Minecraft fabric-loader-0.14.18-1.19.

Stated limitations (Section 4). GPT-4 API cost (15x GPT-3.5); GPT-3.5 and open-source LLMs lack the code-generation quality needed; the agent sometimes gets stuck and cannot produce a correct skill; self-verification errs (e.g. does not accept spider string as proof of killing a spider); curriculum hallucinates impossible tasks (“copper sword”); code generation hallucinates invalid actions (cobblestone as fuel) and non-existent API functions; text-only (no vision).

Use in our design. Take the full admission loop (propose -> generate -> execute -> critic -> store), the description-embedding key/value store with top-5 retrieval, the query = task context + environment feedback, the 4-round retry budget, and the name-collision `V2` versioning. For a single-user local system: replace GPT-4 roles with the senior (curriculum, critic) and the worker (code generation), replace the LLM critic with the verifier cascade (tests first, LLM critic last), replace the OpenAI embedding with a local embedding model, and note the paper’s own finding that a weaker code generator cuts skill discovery 5.7x: this is why admission must be test-gated, not critic-gated.

PDF: https://arxiv.org/pdf/2305.16291 — Code: https://github.com/MineDojo/Voyager

1.2 SoK: Agentic Skills — Beyond Tool Use in LLM Agents (Jiang et al., 2026) — arXiv:2602.20867

Problem and claim. LLM agents re-derive an execution strategy for every task; procedural knowledge dies with the context window. Claim: *skills* (callable procedural modules bundling applicability conditions, an executable policy, termination criteria and an interface) are a distinct abstraction from tools, plans and memory, and should be engineered across a full lifecycle. Systematization of ~180 candidate papers (65 retained, 24 systems mapped), Jan 2020 - Feb 2025 plus SkillsBench (Feb 2026). arXiv preprint (cs.CR), University of Technology Sydney / CSIRO Data61.

Headline results (SkillsBench anchor case study, Section VIII-D; single non-peer-reviewed benchmark, authors call for replication). 86 tasks, 11 domains, 7 agent-model configurations, 7,308 trajectories, deterministic verifiers. Curated skills: **+16.2 pp** average pass rate (24.3% -> 40.6%). By domain: healthcare +51.9 pp, manufacturing +41.9 pp, cybersecurity +23.2 pp, mathematics +6.0 pp, software engineering +4.5 pp. **Self-generated skills: -1.3 pp** on average (only 1 of 5 configurations improved; Claude Opus 4.6 +1.4 pp, Codex/GPT-5.2 -5.6 pp). **2-3 focused skill modules: +18.6 pp; 4+ skills: +5.9 pp.** “Detailed” skills +18.8 pp vs “comprehensive” documentation -2.9 pp. Claude Haiku 4.5 with skills (27.7%) beat Claude Opus 4.5 without (22.0%). 16 of 84 tasks got worse with skills; worst case -39.3 pp (skill guidance conflicted with a model that already did well). ClawHavoc (Section VII-F): 1,184 malicious skills on ClawHub within weeks; 36.8% of audited skills had at least one flaw; 283/3,984 (7.1%) exposed plaintext credentials; 91% of malicious skills carried injected prompts; 12 publisher accounts, one with 677 packages (57%).

Method — equations. Definition 1 (Section II-A):

$$S = (C, \pi, T, R)$$

$$C: O \times G \to \{0,1\}, \qquad \pi: O \times H \to A \cup \Sigma, \qquad T: O \times H \times G \to \{0,1\}, \qquad R = (\text{name}, \text{params}, \text{returns})$$

O observation space, A action space, G goal space, H interaction history up to step t, Σ the skill library. C is the applicability condition (implementations may use a soft score in $[0,1]$ with a threshold); π the executable policy (prompt template, Python function, learned controller or hybrid; selecting a member of Σ instead of a primitive action yields hierarchical composition); T the termination condition (done, successfully or not, relative

to the goal); R the callable contract. C and T parallel the initiation set and termination condition of RL options (I, π, β); R is what makes a skill invocable. In orchestrator-managed systems C and T may be supplied externally.

Method — algorithm (the taxonomies and lifecycle). 1. **Boundaries (Table I).** Tool: single API call, stateless, verified by I/O schema, chained sequentially, governed per-tool permission. Plan: task decomposition, one-time scaffold, step consistency, hierarchical, ephemeral. Episodic memory: stored observation, retrieved not executed, relevance/recency, indirect composability, store access control. Prompt template: text fragment, injected, output quality, string concatenation. Agentic skill: procedural module, callable workflow with termination, verified by outcome correctness and safety, composable hierarchically / as DAG / recursively, governed by trust tier, sandboxing, provenance. 2. **Lifecycle (Fig. 2, Section IV), seven stages:** Discovery -> Practice/Refinement -> Distillation -> Storage -> Retrieval/Composition -> Execution -> Evaluation/Update. Feedback loops: evaluation -> practice (underperforming skill), retrieval -> storage (indexing fails to surface a skill), execution -> discovery (runtime failure reveals a missing skill). Practice improves reliability by iteration (Reflexion, Inner Monologue, Eureka); distillation changes representation (trajectory -> code; prompt -> weights: AgentTuning, FireAct). Storage/retrieval: Voyager (description embedding), CRADLE (skills linked to episodic context), MemGPT (hierarchical memory), Generative Agents (recency/importance/relevance). Evaluation/Update covers “revising, replacing, or retiring” skills but gives no concrete procedure. 3. **Seven design patterns (Section V, Tables III-IV).** P1 Metadata-driven disclosure (name, description, trigger conditions; full instructions load only after selection; Claude Code, Semantic Kernel, LangChain; risk: metadata poisoning). P2 Code-as-skill (Python/shell/DSL programs; composition by function calls, imports, control flow; deterministic, testable; brittle to API change; Voyager, CodeAct, SWE-agent). P3 Workflow enforcement (hard-gated process such as test-first; controller-level; TDD agents, LATS). P4 Self-evolving libraries (after each task decide whether to distill a new skill or refine one; Voyager, DEPS, CRADLE; risk “skill debt”, drift, poisoned distillation). P5 Hybrid NL+code macros (NL spec + executable parts; Claude skills = markdown with instructions, code blocks, asset references; risk: boundary ambiguity). P6 Meta-skills (skills that generate/compose skills by mining task history and testing candidates on held-out tasks; risk: recursive error amplification). P7 Plugin/marketplace distribution (versioned packages with dependency/compatibility/governance metadata; risk: supply chain). Trade-offs (Table IV, context cost / determinism / composability / governance): P1 L/L/M/M; P2 L/H/H/H; P3 M/H/M/H; P4 M/M/M/L; P5 M/M/M/M; P6 H/L/L/L; P7 L/varies/H/M-H. Systems use a median of 2 patterns (range 1-4). 4. **Representation x Scope (Section V-J).** Representation: natural-language skills; code-as-skill; tool macros (parameterized tool-call sequence); policy-based (fine-tuned weights); hybrid. Scope: single-tool; multi-tool orchestration; web (WebArena, Mind2Web); OS/desktop (OSWorld); software engineering (SWE-bench); robotics. Table V: Voyager = P2+P4, code, game/robotics; SWE-agent = P2+P3, code, SWE; OpenClaw = P2+P4+P6+P7, code/hybrid, multi. 5. **Composition and routing (Section VI-F).** Hierarchical: high-level skill invokes mid-level, which invokes low-level, managed by a planning layer that decomposes and routes (HuggingGPT, LATS, MetaGPT, AutoGen). Routing procedure: (i) embedding retrieval of task vs skill descriptions, top-k into context (Voyager, AppAgent); (ii) LLM-mediated routing over metadata with progressive disclosure; (iii) hybrid: embedding narrows, agent picks. No explicit conflict-resolution policy exists (open problem). Failure recovery may itself be a skill, at least as trusted as the skill it recovers. No formal composition algorithm is given. 6. **Security (Section VII).** Threats: poisoned retrieval, malicious payloads, cross-tenant leakage, drift exploitation, confused deputy via environmental injection, applicability-condition poisoning. Four trust tiers: T1 metadata only; T2 instruction access (read-only load, separate approval channel for tools); T3 supervised execution (approval or constrained sandbox); T4 autonomous within preset permissions under monitoring. Default untrusted skills to T1; enforce tiers in the runtime, not in metadata; promote after reliable behavior; demote after one safety violation. Supply chain: provenance signing, dependency auditing, continuous monitoring, version pinning. Code skills need file/network/system sandboxes (Docker, WebAssembly). 7. **Evaluation (Section VIII).** Five dimensions: correctness (ground truth / deterministic verifiers: unit tests, environment-state checks); robustness (input variation, perturbation, edge cases); efficiency (tokens, wall-clock, tool calls, spend); generalization (OOD: cross-site, cross-application); safety (permission boundaries, graceful failure, adversarial tests). Prefer deterministic outcome harnesses over grading reasoning. Benchmarks: SkillsBench (multi: utility, composition, domain effects), WebArena, Mind2Web, OSWorld, SWE-bench (code understanding, patch generation), GAIA, AgentBench, AndroidWorld.

Hyperparameters and setup. Not applicable (survey). Corpus: 6 databases, ~180 candidates, 65 retained, 24 systems mapped, 8 benchmark environments, 7 patterns, 5 representation categories.

Ablations that matter. The SkillsBench numbers above are the only quantitative evidence: curation vs self-generation (+16.2 vs -1.3 pp), skill count (2-3 vs 4+), skill length (detailed vs comprehensive).

Code / data / license. None (survey). SkillsBench and OpenClaw/ClawHub are cited, not released by the authors.

Stated limitations (Section IX-B). Corpus recency (2023-2024 systems), 24 systems from 65 papers, taxonomy not validated by expert survey, production frameworks (LangGraph, DSPy) and safety benchmarks (AgentHarm, InjectAgent) under-represented, reliance on published benchmarks. Open problems (Section X): verified autonomous skill generation (treat skills like CI artifacts, evaluate against held-out task distributions before admission); unsupervised discovery (repeated trajectory patterns, recurring subgoals); formal verification across representations (rule-based for code, semantic inspection for NL, runtime verification); robustness under drift (monitor execution statistics, detect deviation from history, correlate failures with environment change; adapt or retire); governance economics.

Use in our design. Adopt $S = (C, \pi, T, R)$ as the skill record schema (applicability predicate, code, termination/postcondition, contract). Adopt the lifecycle with its three feedback loops as the state machine of the library. Adopt the four trust tiers for self-generated vs user-authored skills, with demotion on a single verifier failure. Take the admission rule from X-A: a skill enters the library only after passing held-out tests. Take the empirical warning: self-generated skills are net negative unless verification-gated and short (2-3 focused skills per task, "detailed" not "comprehensive"). For a single user, cross-tenant and marketplace threats collapse to one: never execute a skill outside the sandbox.

PDF: https://arxiv.org/pdf/2602.20867 — Code: none

1.3 DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning (Ellis et al., 2020) — arXiv:2006.08381

Problem and claim. Program induction needs a hand-written DSL and a hand-designed search; brute-force search is intractable. Claim: learn both. A library L of reusable abstractions (declarative knowledge, the prior over programs) and a neural recognition model Q that guides search (procedural knowledge) are grown jointly by a wake-sleep cycle: wake solves tasks by search, abstraction-sleep compresses solutions into new library routines, dream-sleep trains Q on replays and fantasies. Published at PLDI 2021 as "DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning".

Headline results. List processing: 218 problems (50/50 split, 15 examples each), ~20 library routines learned, rediscovers `filter`. Text editing (SyGuS 2017, 108 test problems, 128 generated training tasks): before learning **3.7%** solved within 10 min (avg 235 s); after learning **79.6%** (avg 40 s); with CVC4's budget (1 h, 8 CPUs) **84.3%** vs CVC4 82.4%. LOGO graphics (160 images, 20 cycles) and towers (107 tasks): no alternative exceeds 60% held-out, DreamCoder near 100%. Physics (60 laws): 93% after 8 cycles, including Newton's and Coulomb's laws. Recursive list programming: all 20 tasks, ~5 days on 64 CPUs, rediscovers `fold`/`unfold`. Across domains: mean solve time 54.1 s, median 15.0 s; library depth vs tasks solved $r = 0.79$. Sorting with learned library: 5 calls vs 32 primitive calls (brute force > 10^{72} years).

Method — equations. Eq. 1 (Section "Wake/Sleep Program Learning", arXiv version):

$$\rho_x = \arg\max_{\rho} P[\rho \mid x, L] \propto P[x \mid \rho]\, P[\rho \mid L] \quad \text{for each } x \in X \qquad \text{(Wake)}$$

$$L = \arg\max_{L} P[L] \prod_{x \in X} \max_{\rho \text{ a refactoring of } \rho_x} P[x \mid \rho]\, P[\rho \mid L] \qquad \text{(Sleep: Abstraction)}$$

$$Q(\rho \mid x) \approx P[\rho \mid x, L], \quad x \sim X \text{ (replay) or } x \sim L \text{ (fantasy)} \qquad \text{(Sleep: Dreaming)}$$

X training tasks; ρ a program; ρ_x the program found for x; L the library defining the generative model $P[\rho \mid L]$; $P[L]$ a description-length prior; $P[x \mid \rho]$ likelihood (0/1 for I/O tasks); Q the recognition network. Equivalent MDL form of abstraction (same section; the paper writes the library as D in this form, $D = L$):

$$\min_{D} (-\log P[D]) + \sum_{x \in X} \min_{\rho \text{ refactors } \rho_x} (-\log P[x \mid \rho]\, P[\rho \mid D])$$

Dreaming objective (MAP training of Q, 50/50 replay/fantasy):

$$\mathbb{E}_x[\log Q(\arg\max_{\rho} P[\rho \mid x, L] \mid x)]$$

From the PLDI 2021 version (open PDF; same system; equation numbers are PLDI's): joint objective and lower bound, Section 2.4, Eqs. 1-3:

$$J(D, \theta) \triangleq P[D, \theta] \prod_{x \in X} \sum_{\rho} P[x \mid \rho]\, P[\rho \mid D, \theta] \qquad (1)$$

$$D^{*} = \arg\max_{D} \int J(D,\theta)\, d\theta, \qquad \theta^{*} = \arg\max_{\theta} J\left(D^{*},\theta\right) \qquad (2)$$

$$\mathcal{L}(D,\theta,\{B_x\}) \triangleq P[D,\theta] \prod_{x\in X} \sum_{\rho\in B_x} P[x\mid\rho]\, P[\rho\mid D,\theta] \le J(D,\theta) \qquad (3)$$

D the library (set of typed lambda-calculus expressions), θ real weights acting like PCFG production probabilities (one per library element, symmetric Dirichlet prior $P[\theta\mid D]$), B_x the beam of programs for task x with $|B_x|\le 5$. Wake maximizes $\mathcal{L}$ over $\{B_x\}$, sleep over D and θ. Abstraction objective with AIC approximation (PLDI Eq. 4):

$$\log P[D] + \max_{\theta} \sum_{x\in X} \log \sum_{\rho\in B_x} P[x\mid\rho] \max_{\rho':\rho'\to_\beta^{*}\rho} P[\rho'\mid D,\theta] + \log P[\theta\mid D] - \|\theta\|_0 \qquad (4)$$

$$P[D] \propto \exp(-\lambda \sum_{\rho\in D} \text{size}(\rho))$$

λ a hyperparameter; $\|\theta\|_0$ number of nonzero continuous degrees of freedom; ρ' ranges over refactorings of ρ (programs beta-reducing to ρ in a bounded number of steps). Refactoring via version spaces (PLDI Eqs. 5-6, $n=3$ in all experiments):

$$\left[\left[I^{n}_{\beta}(\rho)\right]\right] = \{\rho' : \rho' \to_\beta \rho'' \to_\beta \cdots \to_\beta \rho \text{ in at most } n \text{ steps}\} \qquad (5)$$

$$\left[\left[I'_{\beta}(v)\right]\right] = \{\rho' : \rho' \to_\beta \rho \ \forall \rho \in [\,[v]\,]\} \qquad (6)$$

v a version space (de Bruijn index, primitive, lambda, application, union, empty, or all-expressions); $[\,[v]\,]$ its extension; $\text{extract}(v\mid D) = \arg\min_{\rho\in[\,[v]\,]} \text{size}(\rho\mid D)$ where each library member has size 1. Recognition model (PLDI Section 4): $Q(\rho\mid x)$ is a bigram model over syntax trees given by a tensor Q_{ijk}, $i\in D\cup\{\text{var}\}$ the child, $j\in D\cup\{\text{start},\text{var}\}$ the parent, $k\in\{1,\ldots,A\}$ the argument slot; output once per task by a neural net; trained with $\mathcal{L}^{\text{MAP}} = \mathcal{L}^{\text{MAP}}_{\text{Replay}} + \mathcal{L}^{\text{MAP}}_{\text{Fantasy}}$, each the expected log-probability of the MAP program under Q.

Method — algorithm. 1. **Wake.** Sample a minibatch of tasks (or all, if small). For each x, enumerate programs in decreasing $Q(\rho\mid x)$ until a per-task timeout; keep those with $P[x\mid\rho]>0$; store the beam of $k=5$ highest-posterior programs. Search cost is roughly $\text{breadth}^{\text{depth}}$: library learning reduces depth, Q reduces breadth. 2. **Sleep: abstraction.** Hold beams fixed. Build, by dynamic programming with hash-consing and an E-graph, the version space of all refactorings within $n=3$ beta-steps (Figure 3 example: ~10^{14} refactorings in ~10^6 nodes, minutes instead of centuries). Propose candidate primitives from common subexpressions; accept a candidate if it increases the objective (Eq. 4); PLDI: beam search over candidate libraries keeping the top 10^6 that minimize $\text{size}(\text{extract}(v\mid D))$. Repeat until no further increase. 3. **Sleep: dreaming.** Build a training set of 50% replays (waking solutions on real tasks) and 50% fantasies (programs sampled from (D,θ), executed on inputs drawn from the training tasks' input distribution; the target program is found by enumeration in decreasing prior probability, Appendix Algorithm 3 in PLDI). Train Q to maximize the MAP objective, which prefers one canonical simplest solution per task and breaks syntactic symmetries.

Hyperparameters and setup. Beam $k=5$; refactoring bound $n=3$; 50/50 replay/fantasy; 100-200 training tasks; ~1 day on 20-100 CPUs per domain; list and text (PLDI): 12 min per task timeout, 64 CPUs, ~10 cycles; LOGO/towers 20 cycles; physics 8 cycles; Lisp ~5 days on 64 CPUs (~1 CPU-year total); 5 runs per domain. Starting primitives: list: map, fold, cons, car, cdr, ...; LOGO: pen control, imperative control flow, arithmetic on angles/distances; Lisp: 1959 primitives + Y-combinator; symbolic regression: add, mul, div, real parameters fit by inner-loop gradient descent with a BIC penalty. λ numeric value: (not in fetched text — verify in supplement S4.5). Code flags: `-t` enumeration timeout seconds, `-i` wake-sleep iterations, `-RS` recognition steps, `--testingTimeout`.

Ablations that matter. No recognition model: worse at every library depth and shallower final libraries. No abstraction: weaker; library depth correlates with solve rate ($r=0.79$). Exploration-Compression (compression without refactoring), RobustFill-style neural synthesis, enumeration (24 h/task, up to 400 million programs), Memorize (no compression): all below DreamCoder; in LOGO/towers none exceeds 60% held-out vs ~100%.

Code / data / license. https://github.com/ellisk42/ec (MIT; Python + OCaml solvers + Rust compressor). Supplement: https://web.mit.edu/ellisk/www/dreamcodersupplement.pdf (not fetchable here).

Stated limitations (Discussion). Primitives, types, control flow and recognition encoders are built in; learning from a minimal basis is "unlikely to be our best route to AI"; large compute (CPU-year for origami); DreamCoder does not choose or generate its own tasks; handling pervasive noise and uncertainty needs more probabilistic/neural methods.

Use in our design. The consolidation pass is DreamCoder's abstraction sleep applied to a library of LLM-written functions: keep up to 5 verified solutions per task, extract shared sub-functions, and accept a new helper only if the MDL score (library size penalty $\lambda \cdot \text{size}$ plus total size of all skills rewritten to use it) drops. The dreaming phase maps to our weekly LoRA: train the worker on replays (real solved tasks) and fantasies (tasks synthesized by composing library skills and executing them to get ground-truth outputs), 50/50. Replace enumeration-by-Q with the worker sampling candidates ordered by its own log-probability; keep the beam, the refactoring bound and the "one canonical solution" MAP target.

PDF: https://arxiv.org/pdf/2006.08381 — Code: https://github.com/ellisk42/ec

1.4 Executable Code Actions Elicit Better LLM Agents (CodeAct) (Wang et al., 2024) — arXiv:2402.01030

Problem and claim. Agents that act via JSON or text in a fixed format are limited to pre-defined tools, cannot compose tools in one action, and cannot adapt mid-action. Claim: executable Python is a better unified action space: it has pre-training data, native control and data flow, access to existing packages, and built-in error feedback (tracebacks). ICML 2024.

Headline results. - API-Bank, atomic API calls, 17 LLMs (Table 2, % correct, CodeAct / JSON / Text): gpt-4-1106-preview 76.7 / **82.7** / 73.4; gpt-4-0613 75.4 / **82.0** / 74.4; gpt-3.5-turbo-0613 **74.4** / 73.9 / 73.4; claude-2 **76.7** / 59.4 / 73.7; lemur-70b-chat **58.6** / 46.6 / 56.1; Llama-2-70b-chat 35.6 / 14.3 / **37.6**; Llama-2-13b-chat **38.1** / 8.5 / 37.3; Llama-2-7b-chat **28.8** / 11.3 / 25.8; CodeLlama-34b **17.3** / 12.0 / 16.8; CodeLlama-7b 12.5 / 12.0 / **17.0**; Mistral-7B-Instruct-v0.1 2.5 / 2.3 / **3.0**. Best-format counts: CodeAct 8, JSON 5, Text 4 (open-source 4/0/4). - M3ToolEval, 82 human-curated multi-step tasks, 5 domains (web browsing, finance, travel planning, science, information processing), zero-shot, 10-turn cap, exact match (Table 3, success % / avg turns): gpt-4-1106-preview **74.4 / 5.5** (JSON 52.4 / 7.6, Text 53.7 / 7.7); gpt-4-0613 **67.1 / 6.6** (56.1 / 7.6, 45.1 / 8.0); claude-2 **54.9 / 7.2**; gpt-3.5-turbo-0613 **51.2 / 7.0** (26.8, 20.7); gemini-pro 22.0; lemur-70b 13.4 (JSON 15.9); Llama-2-70b 11.0 (3.7, 3.7); Mistral-7B 0.0 (JSON 3.7). CodeAct wins success on 12/17 models and turns on 12/17; up to **20 points** absolute and up to **30%** fewer actions (gpt-4-1106: 5.5 vs 7.7 turns = 29%; the text says 2.1 fewer turns, the table gives 2.2). - CodeActAgent (Table 5, 7B, full SFT): Mistral: MINT ID 57.4, MINT OOD 32.4, M3ToolEval 12.2, MiniWob++ 46.2, ScienceWorld 15.9, MMLU 59.1, HumanEval 34.7, GSM8K 58.0, MTBench 8.2, overall 42.5. Llama-2: 51.3 / 20.4 / 0.0 / 25.5 / 17.6 / 50.6 / 18.1 / 38.3 / 7.5 / 30.7. AgentLM-7B: 8.7 / 6.1 / 0.0 / 28.9 / 13.7 / 48.7 / 15.4 / 24.6 / 6.1 / 24.8.

Method — equations. None.

Method — algorithm. 1. **Framework (Section 2.1).** Three parties: agent, user, environment. The agent emits an action; a CodeAct action is a Python snippet executed by an interpreter; the execution result (stdout or traceback) is returned as the observation; the loop continues multi-turn. 2. **Action-space argument (Table 1).** CodeAct vs Text/JSON: data availability (code in pre-training vs curated format data); complex operations (native loops/conditionals/variables vs engineering new tools to mimic if-statements); tool availability (import existing packages vs curating tools); automated feedback (tracebacks vs hand-routed feedback). 3. **CodeActInstruct (Section 3.1, Table 4).** Sources: information seeking (HotpotQA, 1,664), software packages (MATH 1,732; APPS 647), external memory (WikiTableQuestion, SQL and Pandas variants, 1,065), robot planning (ALFWorld, 2,031); total **7,139** instances, 10,581,681 tokens. Generators: gpt-3.5-turbo-0613 (16k variant for code), claude-1-instant, claude-2; gpt-4-0613 on problems the others failed. Filter: keep trajectories where "the model initially encounters errors but rectifies these inaccuracies in later interactions"; drop malformed actions and trajectories that ignore the user; 6,728 of 6,985 (gpt-3.5/Claude) and 411 of 413 (gpt-4) kept. General mix: OpenOrca CoT 50,000; ShareGPT 10,000; ShareGPT-GPT4 4,583; CapyBara 4,647 (69,230 instances, 55,146,326 tokens). 4. **Execution sandbox (repo).** One Docker container per chat session, JupyterKernelGateway engine, session timeout.

Hyperparameters and setup (Appendix D). 4x A100 40GB, ~9k tokens/s, Megatron-LLM fork, chatML, loss on assistant turns only, packing, flash attention, tensor parallel 4, lr 1e-5, 50 warmup steps, cosine to 1e-6, 5 epochs, batch 32, 3rd-epoch checkpoint used. Sequence length: (not in fetched text — verify in Appendix D).

Ablations that matter (Table A.8, same data in text form). Mistral: CodeActAgent 57.4 / 32.4 MINT ID/OOD vs w/o CodeAct 32.9 / 23.0; MiniWob++ 46.2 vs 47.8; general capabilities unchanged (MMLU 59.1 vs 59.9; HumanEval 34.7 vs 33.2; GSM8K 58.0 vs 59.5). Llama-2: MINT ID 51.3 vs 17.0. Appendix H: Llama-2 CodeActAgent gains 0 on M3ToolEval (unexplained outputs attributed to the pre-training corpus); Mistral gains >10 points.

Code / data / license. https://github.com/xingyaoww/code-act (MIT); HF model `xingyaoww/CodeActAgent-Mistral-7b-v0.1` (32k context), `CodeActAgent-Llama-7b`; dataset `CodeActInstruct` under `xingyaoww/code-act`.

Stated limitations. Llama-2 CodeActAgent anomaly on M3ToolEval, attributed to its pre-training corpus or weaker base capability (Appendix H); JSON still wins atomic calls on several closed models (Table 2). A dedicated limitations section: (not in fetched text — verify in paper Section 5 / Appendix).

Use in our design. Skills are Python functions and skill *invocation* is a CodeAct action: the worker writes code that imports and calls library skills, composes them with loops/conditionals, and reads tracebacks as the first verifier stage. The 7B fine-tuning recipe (full SFT, loss on assistant turns, keep only self-corrected trajectories) is the template for the weekly LoRA data filter. For a single user: one persistent sandbox per session with the library mounted read-only; log every traceback as a candidate "lesson" for the Notebook (pack A).

PDF: https://arxiv.org/pdf/2402.01030 — Code: https://github.com/xingyaoww/code-act

1.5 Large Language Models as Tool Makers (Cai et al., 2023) — arXiv:2305.17126

Problem and claim. Tool-augmented LLMs depend on suitable tools existing; serving every request with a strong model is expensive. Claim (LATM): a strong model (tool maker) writes a reusable Python function for a *class* of requests, verified on held-out examples; a cheap model (tool user) applies it per instance. Tool-making cost is amortized; cached tools form a *functional cache* keyed by request class rather than by response text. ICLR 2024.

Headline results (Table 2, accuracy %, 240 test instances per task).

Task	GPT-3.5 CoT	GPT-3.5 + LATM	GPT-4 CoT	GPT-4 + LATM
Logical Deduction (5)	66.4	79.7 (+13.3)	88.8	86.6
Tracking Shuffled Objects (5)	61.6	99.6 (+38.0)	100.0	100.0
Dyck Language	20.4	92.2 (+71.8)	63.6	87.5
Word Sorting	59.2	98.3 (+39.1)	90.9	99.1
Chinese Remainder Theorem	0.0	100.0 (+100.0)	0.0	100.0
Schedule Meeting	18.9	100.0 (+81.1)	55.6	100.0

Cost: GPT-3.5 CoT $O(nc)$; GPT-3.5 LATM $O(nc+C)$; GPT-4 $O(nC)$, with C (one GPT-4 call) $>15c$. Tool-making success out of 5 trials (Table 3): GPT-4 3/5, 4/5, 5/5, 5/5, 5/5, 3/5; GPT-3.5 0/5, 0/5, 5/5, 5/5, 5/5, 0/5. Dispatcher (GPT-3.5): **95% +- 2%** identifying an existing tool (100 mixed samples, 5 constructions), **96% +- 3%** deciding to request a new tool. Only 2 of 60 tool-making trials used error messages to correct mistakes. Reusing GPT-4 CoT as context for GPT-3.5 (ablation): 36.8 (LD) and 63.2 (TSO) vs LATM 79.7 and 99.6.

Method — equations. Cost accounting only: per-request cost of LATM is $c+C/n$ for n requests of one class (derived from the $O(nc+C)$ statement; c cost of a tool-user call, C of a tool-maker call).

Method — algorithm (Section 3; code in `toolmaker.ipynb`, `tooluser.ipynb`). 1. **Tool proposing.** Input: $k=3$ demonstrations (input-output pairs) of the task class. Tool maker (GPT-4, temperature 0.3) is asked for "a generic Python function to solve this type of problems using only standard python libraries", in a code block. If the code fails to execute, the error is appended to the history and the model retries. 2. **Tool verification.** 3 validation samples. The tool maker writes unit tests (function calls + expected outputs) and runs them. On failure the error is logged and the model fixes **the function calls in the tests, not the function**. Max 3 retries over proposing + verification; beyond the threshold the task is marked failed. 3. **Tool wrapping.** Package = function code + demonstrations of converting a natural-language question into a function call (taken from the verification tests). 4. **Tool using.** Tool user (GPT-3.5, temperature 0.0) receives the wrapped tool and a new question; writes one function call by in-context learning; the call is executed; optional post-processing (e.g. pick a multiple-choice option). One LLM call per instance. 5. **Dispatcher (Section 4, Appendix A).** Maintains a cache of tools exposed as function APIs. For each request: if a compatible cached tool exists, forward request + tool to the tool user; else treat as a new task class, solve with the strong model (or a human), store the instance, and trigger tool making once enough instances of the class have accumulated.

Hyperparameters and setup. 3 train / 3 validation / 240 test per task; 5 BIG-bench tasks + Schedule Meeting (authors'); GPT-4 and GPT-3.5 Turbo via ChatCompletion; temperature 0.3 making, 0.0 using; max 3 retries. Tool-user comparison (Table 4): GPT-3.5 Turbo best trade-off (cost $0.002 / 1k tokens); text-davinci-002 58.2 / 100.0 / 35.8 / 60.8 / 100.0 / 100.0; davinci 11.6 / 62.1 / 16.4 / 26.6 / 99.6 / 62.9.

Ablations that matter. Tool-maker capacity (Table 3): GPT-3.5 fails all 5 trials on the three hardest classes. Tool-user capacity (Table 4): pre-instruction-tuned GPT-3 models beat their instruction-tuned counterparts. CoT reuse does not substitute for a tool.

Code / data / license. https://github.com/ctlllll/LLM-ToolMaker (license not stated on the repo page); `tools/` holds wrapped tools, `bbh/` the BIG-bench JSON, `cot-prompts/`.

Stated limitations (Section 6, Appendix B). No realistic datasets of recurring task streams; tools may be incorrect or harmful; autonomy reduces control; safety not examined in depth; future work: let the tool maker refine existing tools.

Use in our design. This is the admission contract: a skill is admitted only with (a) the function, (b) >= 3 demonstrations as unit tests that passed, (c) call-conversion examples stored next to it for the worker. Senior = tool maker, worker = tool user, dispatcher = a cheap classifier over the library (embedding retrieval first, worker confirms). The “accumulate instances, then make a tool” rule is the trigger for skill creation in a single-user system: create a skill when the same request class recurs m times (choose m; the paper does not give one).

PDF: https://arxiv.org/pdf/2305.17126 — Code: https://github.com/ctlllll/LLM-ToolMaker

2. Design extraction for the module

2.1 Algorithms to implement (consolidated pseudocode)

A. Skill admission (Voyager loop + LATM verification + SoK trust tiers)

```
admit_skill(task, trajectory):
    # trajectory: the verified worker/senior run that solved `task` (tests passed)
    code = extract_main_function(trajectory)             # one Python function, generic, params explicit
(Voyager A.4.2 guidelines)
    demos = collect_io_pairs(trajectory, task)            # >= 3 input/output pairs (LATM: 3 train)
    held_out = sample_or_synthesize_cases(task, n=3)      # LATM: 3 validation; SoK X-A: held-out before
admission
    for attempt in 1..3:                                  # LATM max retries 3; Voyager 4 rounds
        tests = senior.write_unit_tests(code, held_out)   # LATM: fix the calls, not the function
        result = sandbox.run(code, tests)                 # CodeAct: traceback is the first verifier
        if result.all_pass: break
        code = senior.repair(code, result.errors)
    if not result.all_pass: return REJECT
    desc = worker.describe(code)                          # Voyager Prompt 5: <= 6 sentences, no function
name, main function only
    sig  = parse_signature(code)                          # R = (name, params, returns)  (SoK)
    rec  = SkillRecord(name, desc, code, sig, tests, demos, trust_tier=3, version=1, provenance=task.id)
    if name in library: rec.version = library[name].version + 1; rec.name = f"{name}V{rec.version}"  #
Voyager V2 suffix
    rec.embedding = embed(desc)
    library.insert(rec); vectordb.add(id=rec.name, text=desc, meta={"name": rec.name})
    return rec
```

B. Retrieval and prompt injection (Voyager 2.2 + code)

```
retrieve(task, context, env_feedback, k=5):
    query = context + "\n\n" + summarize(env_feedback)     # Voyager voyager.py: context + chatlog summary
    hits  = vectordb.similarity_search_with_score(embed(query), k=min(k, vectordb.count()))
    return [library[h.id] for h in hits]                   # code + signature + description go into the
prompt
# Prompt order (Voyager A.4.1): guidelines, primitives, retrieved skills, last code, env feedback, errors,
critique, state, task, context, CoT.
# Response format: Explain / Plan / Code.
```

C. Dispatcher / planner (LATM Section 4 + SoK VI-F routing)

```
dispatch(request):
    cands = retrieve(request, context="", env_feedback="", k=5)     # embedding narrows (SoK routing iii)
    choice = worker.select(request, [c.desc + c.sig for c in cands]) # worker confirms or says NONE (LATM:
95% / 96%)
    if choice != NONE: return worker.write_call(request, choice)    # one call; CodeAct action composes
several skills
    pending[request.class_key].append(request)
    if len(pending[request.class_key]) >= m: schedule(admit_skill)   # LATM: accumulate, then make a tool
    return senior.solve(request)                                     # fall back to the strong model
```

D. Consolidation at idle time (DreamCoder abstraction sleep, adapted)

```
consolidate(library, beams, lam):
    # beams[task] = up to 5 verified solutions per task (DreamCoder k = 5)
    repeat:
        cands = propose_common_subfunctions(beams, max_depth=3)        # bounded refactoring (n = 3)
        best = None
        for c in cands:
            L2 = library + [c]
            score = lam * sum(size(s) for s in L2) + sum(min(size(rewrite(p, L2)) for p in beams[t]) for t
in beams)
            if score < current_score(library) and all_tests_pass(rewrite_all(beams, L2)): best =
argmin(score)
        if best is None: break
        library = library + [best]; rewrite beams to call best
    # retirement (SoK X-D, VII-B): demote a skill one tier on any verifier failure; retire when
    # success_rate over the last N calls drops below threshold or its tests fail after an environment
change.
    # dedup: merge skills whose descriptions have cosine similarity > tau and whose tests pass on each
other's code.
```

E. Dreaming for the weekly LoRA (DreamCoder dreaming, adapted)

```
build_training_set(library, solved_tasks):
    replays   = [(t, canonical_solution(t)) for t in solved_tasks]         # 50%
    fantasies = []
    while len(fantasies) < len(replays):
        prog = compose_random(library, depth<=3); inputs = sample_inputs(solved_tasks)
        out = sandbox.run(prog, inputs)                                      # execution gives ground truth
        fantasies.append((make_task(inputs, out), prog))
    return replays + fantasies                                               # train worker on MAP (one
canonical program per task)
```

2.2 Equations and quantities the implementation must compute

- Cosine similarity of description embeddings for retrieval; top-k with $k = \min(5, |\Sigma|)$ (Voyager).
- Skill tuple $S = (C, \pi, T, R)$; C as a soft score in $[0,1]$ with a threshold (SoK II-A); in practice C = retrieval score x worker confirmation.
- MDL consolidation score: $\lambda \sum_{\rho \in D} \text{size}(\rho) + \sum_{x} \min_{\rho' \text{ refactors } \rho_x} \text{size}(\rho' \mid D)$ with each library call counting size 1 (DreamCoder / PLDI Eq. 4 and the extract rule); accept a helper only if the score decreases.
- Amortized cost of a skill class: $c + C/n$ (LATM); use it to decide whether making a skill pays: make when expected $n \cdot (C - c) > C$, i.e. $n > C/(C - c)$.
- Admission statistics to log per skill: pass rate on held-out tests, number of calls, success rate of calls, tokens and wall-clock per call (SoK VIII-A efficiency), last-verified timestamp.
- Benchmark deltas in percentage points: curated vs none, self-generated vs none (SkillsBench via SoK): the module is working if self-generated skills are >= 0 pp, not the -1.3 pp reported for ungated generation.

2.3 Data structures and file formats (as TypeScript-style type sketches)

```
        type SkillRecord = {
          name: string;                // R.name; collisions -> nameV2, nameV3 (Voyager)
          version: number;
          description: string;         // <= 6 sentences, single line, no function name (Voyager Prompt 5)
          code: string;                // one Python function `def name(...)`; helpers inside (Voyager
A.4.2 / CodeAct)
          signature: { params: { name: string; type: string; doc?: string }[]; returns: { type: string;
doc?: string } }; // R
          applicability: string;       // C: NL predicate or small code check; soft score threshold in
[0,1] (SoK)
          termination: string;         // T: postcondition / success check used by the verifier (SoK)
          tests: { call: string; expected: string }[];      // >= 3 held-out cases that passed (LATM
verification)
          demos: { question: string; call: string }[];      // NL -> function-call conversions (LATM
wrapping)
          embedding: number[];         // of `description`
          trustTier: 1 | 2 | 3 | 4;    // SoK VII-B; self-generated starts at 3 (supervised/sandboxed),
user-authored may be 4
          provenance: { taskId: string; author: "worker" | "senior" | "user"; createdAt: string;
parentSkills: string[] };
          stats: { calls: number; successes: number; lastVerifiedAt: string; avgTokens: number; avgMs:
number };
          status: "active" | "deprecated" | "retired";
        };

        type SkillLibraryOnDisk = {
          "skill/code/<name>.py": string;          // Voyager layout: code/, description/, vectordb/,
skills.json
          "skill/description/<name>.txt": string;
          "skill/tests/<name>.json": SkillRecord["tests"];
          "skill/skills.json": Record<string, Pick<SkillRecord, "code" | "description" | "version" |
"trustTier" | "status">>;
          "skill/vectordb/": unknown;              // persisted vector index, ids = skill names
        };

        type Beam = { taskId: string; solutions: { code: string; sizeTokens: number; passed: true }[] /*
|solutions| <= 5 */ };

        type DispatchRequest = { text: string; classKey: string; context: string; envFeedback: string[] };
        type PendingClass = { classKey: string; instances: DispatchRequest[]; threshold: number /* m */ };

        type ConsolidationLog = { added: string[]; merged: [string, string][]; retired: string[];
scoreBefore: number; scoreAfter: number };
```

2.4 Hyperparameters and defaults to start from (with the paper each comes from)

Quantity	Default	Source
Retrieval top-k	5	Voyager Section 3.1, `retrieval_top_k=5`
Code-generation retries per task	4	Voyager A.1, `action_agent_task_max_retries=4`
Tool-making retries	3	LATM Section 5.1
Demonstrations / validation cases per skill	3 / 3	LATM Section 5.1 (`k = 3`)
Temperature: making / using / curriculum	0.3 / 0.0 / 0.1	LATM Section 5.1; Voyager Section 3.1 (all other calls 0)
Description length	<= 6 sentences, one line	Voyager Prompt 5
Skills injected per task	2-3 focused	SoK VIII-D (SkillsBench: +18.6 pp vs +5.9 pp for 4+)
Beam of solutions kept per task	5	DreamCoder (k = 5)
Refactoring depth bound	3	DreamCoder (n = 3)
Replay : fantasy ratio for training	50 : 50	DreamCoder dreaming

Quantity	Default	Source
Library size penalty λ	tune; (not in fetched text)	DreamCoder PLDI Eq. 4
Trust tier for new self-generated skills	3 (supervised/sandboxed); demote on one violation	SoK VII-B
Curriculum warm-up (state fields revealed by completed tasks)	0 / 5 / 7 / 10 / 15	Voyager Table A.1
SFT recipe for the worker	lr 1e-5, cosine to 1e-6, 50 warmup, 5 epochs, batch 32, loss on assistant turns, use epoch 3	CodeAct Appendix D
Trajectory filter for training	keep runs that erred then self-corrected	CodeAct Section 3.1
Max agent turns per task in evaluation	10	CodeAct M3ToolEval

2.5 Evaluation: how to know the module works (metrics, from the papers)

- **Three-arm held-out comparison** (SkillsBench via SoK VIII-D): same tasks with no skills / curated skills / self-generated skills; report pass-rate deltas in pp per domain; target self-generated >= 0 pp (paper: -1.3 pp ungated) and curated around +16 pp.
- **Admission precision**: fraction of admitted skills whose held-out tests still pass on re-run (LATM verification); tool-making success per class out of 5 trials (LATM Table 3).
- **Dispatcher accuracy**: existing-skill identification and new-skill request accuracy on 100-sample mixes, 5 constructions (LATM: 95% +- 2%, 96% +- 3%).
- **Compositional growth**: unique task types solved per iteration and iterations-to-milestone with and without the library (Voyager Table 1: iron 21 +- 7 vs 29 +- 11; zero-shot transfer 18-21 vs 26-36 iterations).
- **Turn and action efficiency**: average turns per solved task with code actions vs JSON/text (CodeAct Table 3: 5.5 vs 7.7).
- **Library depth vs solve rate**: correlation across consolidation cycles (DreamCoder $r = 0.79$); mean/median solve time after learning (54.1 s / 15.0 s; text editing 235 s -> 40 s).
- **Five SoK dimensions** per skill: correctness (deterministic verifier), robustness (perturbed inputs), efficiency (tokens, ms, calls), generalization (OOD tasks), safety (sandbox violations = 0).

2.6 Known pitfalls and failure modes (from the papers' limitations)

- Weak code generators collapse skill discovery (Voyager: GPT-3.5 yields 5.7x fewer items; Llama-2 CodeActAgent gains 0 on M3ToolEval): gate admission on tests, route hard skill-making to the senior.
- LLM critics misjudge success (Voyager: spider string example) and curricula hallucinate impossible tasks ("copper sword"): never admit on critic verdict alone; require executable evidence.
- Hallucinated API calls and invalid operations (Voyager Section 4): import-check and static-analyze skill code before running tests.
- Self-generated skills are net negative without curation (SoK: -1.3 pp; 16/84 tasks worse, up to -39.3 pp when guidance conflicts with a capable base): inject at most 2-3 skills; prefer short, detailed skills over comprehensive docs; measure per-task harm and retire.
- Skill debt and drift (SoK V-E, X-D): skills break when APIs or environments change; re-run tests on a schedule, demote on failure, correlate failures with environment changes.
- Recursive error amplification in meta-skills (SoK P6): cap generation depth, quality-gate every generated skill.
- Error messages rarely rescue a bad tool (LATM: 2 of 60 trials): budget retries at 3, then escalate.
- JSON can still beat code for atomic single calls on strong closed models (CodeAct Table 2): keep a trivial-call fast path.
- Compute: DreamCoder-style refactoring is polynomial in program size but exponential in the step bound; keep $n \leq 3$ and run only at idle time.
- Security: code skills need a sandbox with file/network/system limits (SoK VII-C; CodeAct uses one Docker container per session); untrusted skills default to metadata-only.

2.7 Open questions this section does not settle

- The value of λ (library size penalty) and the exact candidate-proposal procedure for LLM-written Python (DreamCoder's version-space refactoring is defined for typed lambda calculus; a Python analogue via AST subtree mining is our design, not the paper's).
- The accumulation threshold m before a request class triggers skill-making (LATM gives none).
- Conflict resolution when several skills apply (SoK VI-F: open problem; no system has a policy).
- Whether a 9B worker can reliably write the function *call* from the wrapped-tool demos (LATM shows GPT-3.5 can; nothing in the fetched text covers a 9B local model).
- How to define "task class" keys for the dispatcher cache in open-ended coding work (LATM used six fixed BIG-bench classes).
- Retirement thresholds (success-rate floor, re-verification period): SoK names the mechanism but gives no numbers.
- How the DreamCoder recognition model's bigram parameterization maps onto an LLM's own token probabilities for ordering candidate programs (we assume the worker's log-probability; not validated in any fetched paper).
- The definition of a "prompting iteration" in Voyager (not stated in the fetched text), needed if its iteration counts are used as targets.

3. Verified source table

arXiv id	Title	Year	Venue	Code	PDF
2305.16291	Voyager: An Open-Ended Embodied Agent with Large Language Models	2023	TMLR 2024 (arXiv v2 Oct 2023)	https://github.com/MineDojo/Voyager (MIT)	https://arxiv.org/pdf/2305.16291
2602.20867	SoK: Agentic Skills — Beyond Tool Use in LLM Agents	2026	arXiv preprint (cs.CR), v1 24 Feb 2026	none	https://arxiv.org/pdf/2602.20867
2006.08381	DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning	2020	arXiv 2020; PLDI 2021 version "DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning"	https://github.com/ellisk42/ec (MIT)	https://arxiv.org/pdf/2006.08381
2402.01030	Executable Code Actions Elicit Better LLM Agents	2024	ICML 2024 (arXiv v4 Jun 2024)	https://github.com/xingyaoww/code-act (MIT)	https://arxiv.org/pdf/2402.01030
2305.17126	Large Language Models as Tool Makers	2023	ICLR 2024 (arXiv v2 Mar 2024)	https://github.com/ctlllll/LLM-ToolMaker (license not stated)	https://arxiv.org/pdf/2305.17126